

“Solace – A Privacy-First End-to-End Encrypted Chat Platform with Team Workspaces”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree
of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

HRUSHIKA HABEEB

2520090054

AMRUTHA J

2520080066

CHAKRIKA PANAKALA

2520090042

Under the Esteemed Guidance of

Dr. Sk. Kaja Shareef

Head of the Department

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Solace – A Privacy-First End-to-End Encrypted Chat Platform with Team Workspaces”** is a record of bonafide work of **HRUSHIKA HABEEB – 2520090054**, **AMRUTHA J – 2520080066**, **CHAKRIKA PANAKALA – 2520090042** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 14. during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

HRUSHIKA HABEEB – 2520090054
AMRUTHA VARSHINI J– 2520080066
CHAKRIKA PANAKALA – 2520090042

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“Solace – A Privacy-First End-to-End Encrypted Chat Platform with Team Workspaces”** is a bonafide work done and submitted by **HRUSHIKA HABEEB – 2520090054, AMRUTHA J – 2520080066, CHAKRIKA PANAKALA – 2520090042** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 12, Team 14, at KLH University during 2026–2027, Trimester 4.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to everyone who assisted us in one way or another in completing this project.

Our greatest appreciation goes to our Course Coordinator, **Ramatenki Sateesh Kumar**, and to our guide, **Dr. Sk. Kaja Shareef**, Head of the Department of Computer Science and Information Technology, for his tremendous support, encouragement and guidance throughout this project.

We express our gratitude to **Dr. Sk. Kaja Shareef**, Head of the *Department of Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

HRUSHIKA HABEEB – 2520090054

AMRUTHA J – 2520080066

CHAKRIKA PANAKALA – 2520090042

Abstract

Solace is a privacy-first web chat platform designed so that nobody but the intended participants — including the operator of the server — can read what people say. Conventional messaging services store messages in a form the provider can read, and work coordination is scattered across separate tools for tasks, calendars and files. Solace addresses both problems in one application: every message, file and voice note is encrypted in the sender's browser before it is sent, and any group conversation can be turned into a Workspace with a shared task board, calendar, notes, decision log, polls and file library.

The system provides real-time one-to-one and group messaging over WebSocket, with delivery and read receipts, typing indicators, edit, delete, reply, forward, reactions, pins, stars, link previews, encrypted media and voice messages, disappearing and scheduled messages, blocking and reporting, and peer-to-peer voice and video calls whose signaling is also end-to-end encrypted. Encryption uses only standard WebCrypto primitives (ECDH P-256, HKDF-SHA256 and AES-256-GCM) in a stateless multi-recipient envelope, so a message can be decrypted any number of times, in any order and after a reload, and group membership changes need no key rotation. The server stores and routes ciphertext, public keys and routing metadata only.

The application uses a Next.js 14 and React frontend written in TypeScript, a Fastify REST API with a WebSocket hub, and MongoDB (run as a replica set) as its only datastore: Prisma holds the core models, the native driver holds notes, decisions, polls and call state, GridFS holds encrypted media, and change streams provide real-time fan-out. Passwords are hashed with Argon2id, and sessions use short-lived JWT access tokens with rotating refresh tokens. The system was built and verified in incremental phases, and is backed by 60 automated tests and a continuous-integration pipeline. The result is a secure, self-hostable platform that keeps conversations private while giving teams the tools to organise their work. The application is deployed and can be accessed at <https://solace-lime-eight.vercel.app/>.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	4
		Abstract	5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	8
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	10
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	11
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	12
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions 5.4 End-to-End Encryption Design	14
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	16
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	18
8	Testing	8.1 Automated Unit and Integration Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	19
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	21
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	22
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	23
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	24
13	References	Documentation, tools, and resources used	25

S.No.	Chapters	Topics	Page No.
14	Appendices	Sample Code Snippets Installation / Setup Instructions Default Login Credentials API Reference Summary Screenshots (to be added by the student)	26

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Instant messaging has become the default way students, teams and families communicate. Most popular platforms, however, rely on the service provider's servers to store messages in a form the provider can read, which creates a single point of privacy failure: a database breach, an insider, or a legal request can expose every conversation. At the same time, work-related chat is usually split across several tools — one for messaging, another for tasks, another for calendars and another for file sharing — so the context of a conversation is lost as soon as work moves out of the chat window.

Solace was built to address both problems in a single web application. It is a privacy-first messaging platform in which every message, file and voice note is encrypted in the sender's browser before it is sent, so the server stores and routes ciphertext only. On top of the messaging core, any group conversation can be turned into a Workspace that adds a shared task board, calendar, notes, decision log, polls and file library, all protected by the same encryption boundary.

1.2 Problem Statement

Given a population of users who need to exchange text, media and voice or video calls in real time, and teams that need to coordinate work around those conversations, the system must answer three questions reliably: how can messages be delivered instantly while ensuring that the server operator can never read them; how can group membership change (members added or removed) without exposing past or future content to the wrong people; and how can collaboration features such as tasks, events and polls live next to the chat without weakening its confidentiality. Conventional chat applications trade one of these properties away; Solace is designed to keep all three.

1.3 Scope of the Project

The system covers registration and login with device-bound encryption keys; one-to-one and group messaging with end-to-end encryption; message features (edit, delete, reply, forward, reactions, pins, stars, mentions, search and link previews); encrypted images, videos, files and voice messages; disappearing and scheduled messages; blocking and reporting; one-to-one voice and video calls over WebRTC; and the Workspace layer (tasks with subtasks and comments, calendar events with RSVP, notes, a decisions log, polls, and a file library with folders). Group voice or video calls, a native mobile application, per-message forward secrecy, and federation with other messaging networks were treated as out of scope for this submission.

1.4 Objectives

- Provide real-time one-to-one and group messaging over WebSocket with delivery and read receipts and typing indicators.
- Guarantee that the server never receives message plaintext, unencrypted media, or any private key, using only standard WebCrypto primitives (ECDH P-256, HKDF-SHA256, AES-256-GCM) with no custom cryptography.
- Make group membership changes safe without key-distribution rounds: a removed member cannot read anything sent after removal.

- Add a collaboration layer (tasks, events, notes, decisions, polls, files) to any group while keeping content fields encrypted.
- Support peer-to-peer voice and video calls whose signaling is itself end-to-end encrypted.
- Secure accounts with Argon2id password hashing, short-lived JWT access tokens, rotating refresh tokens in HttpOnly cookies, and rate limiting.
- Use a single datastore (MongoDB) so the whole system is simple to install, run and deploy.

1.5 Importance of the Application

Privacy is increasingly treated as a baseline requirement rather than a premium feature. By making encryption the default and keeping the server blind to content, Solace removes the need for users to trust the operator. Teams gain a single place where a conversation can immediately become a task, an event or a poll, which reduces context switching. For the authors, the project is also a practical study of applied cryptography, real-time distributed systems and full-stack TypeScript engineering.

1.6 Target Users / Audience

The system serves individuals and small teams — student project groups, clubs, research teams and small organisations — who want a private chat that also helps them organise work. Within a group, three roles exist: Owner, Admin and Member. Owners and Admins manage membership and workspace settings, while all members can message, create tasks and events, vote in polls and share files.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application is lightweight enough to run the complete stack — web client, API server and MongoDB — on one development machine. A dual-core processor, 4 GB of RAM (8 GB recommended when the Next.js development server, MongoDB and a browser are all running) and about 2 GB of free disk space for Node modules, the Next.js build cache and MongoDB data files are sufficient. A working microphone and camera are needed only to try voice messages and calls.

2.2 Software Requirements

- Node.js 20 or later, with npm (workspaces support)
- MongoDB 5.0 or later, running as a replica set (a single-node replica set is enough), or a free MongoDB Atlas cluster
- A modern web browser with WebCrypto, IndexedDB and WebRTC support (Chrome, Edge, Firefox or Safari)
- Git, for version control
- MongoDB Compass / mongosh and Visual Studio Code, for development (optional but recommended)
- coturn (TURN relay) and Caddy (reverse proxy), only for production-style deployments

2.3 Development Tools and Frameworks

The web client was built with Next.js 14 (App Router), React 18, TypeScript and Tailwind CSS. The API server uses Fastify 4 with TypeScript, Zod for request validation, and Prisma 5 (MongoDB provider) together with the native MongoDB driver. The shared cryptography library is a TypeScript workspace package that depends only on the browser's WebCrypto API. Tests use the Node.js built-in test runner executed through tsx, with fake-indexeddb and mongodb-memory-server as test helpers. Git and GitHub (with GitHub Actions for continuous integration) were used for source control, and VS Code was the primary editor.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a Next.js 14 application using the App Router, written in TypeScript and styled with Tailwind CSS. Server-side rendering is used only for the public landing and authentication routes; the chat itself (/chat) is a client-rendered application because every piece of content must be decrypted in the browser. A shared design-token package (CSS variables plus a Tailwind preset) defines the lavender, blush and coral theme, and the Fraunces and Inter typefaces are loaded through Google Fonts. State is held in React state and context (for example the authentication session), with small purpose-built modules for realtime sockets, drafts, media pipelines and call handling.

3.2 Backend

The backend is a Fastify 4 REST API written in TypeScript. Each feature area has its own route module (auth, keys, conversations, groups, message actions, media, workspaces, tasks, events, notes, decisions, polls, files, calls, moderation, link preview and GIF search), and request bodies are validated with Zod schemas before any business logic runs. A native WebSocket endpoint (/ws, via @fastify/websocket) delivers messages, receipts, typing indicators and call signals in real time. Background sweep jobs expire disappearing messages and deliver scheduled ones. Compression and global rate limiting are applied through Fastify plugins.

3.3 Database

MongoDB, run as a replica set, is the only datastore. Core relational-style models (users, devices, conversations, memberships, messages, receipts, media metadata, reactions, pins, stars, blocks, reports, workspaces, tasks, subtasks, events, RSVPs and folders) are defined once in a Prisma schema and accessed through Prisma's MongoDB provider. Flexible, document-shaped content (notes, decisions, polls and task activity) uses the native driver. Encrypted media blobs are stored in a GridFS bucket named media. Realtime fan-out between server processes uses change streams on a realtime_events collection, and call state (sessions, busy locks, rate limits) lives in TTL-indexed collections so stale entries expire by themselves.

3.4 Version Control

The project is a single Git repository organised as an npm-workspaces monorepo containing apps/web, apps/server, packages/crypto and packages/design-tokens. A GitHub Actions workflow installs dependencies, generates the Prisma client, runs the crypto and server test suites, type-checks the server, and builds the web application on every push to main and on every pull request.

3.5 Authentication and Security Libraries

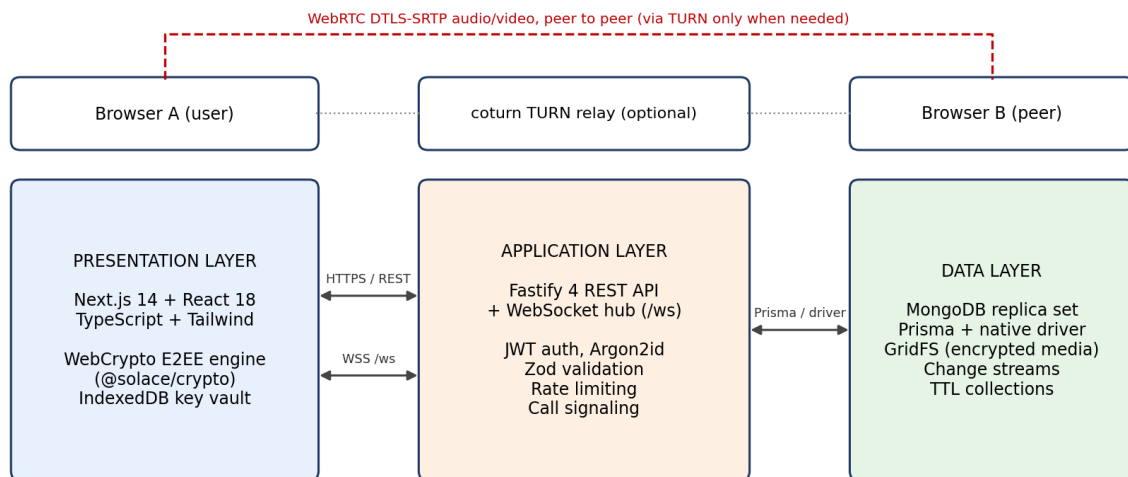
Passwords are hashed with Argon2id, the current OWASP recommendation. Sessions use a short-lived JWT access token (jsonwebtoken) plus a rotating refresh token stored in an HttpOnly cookie, and the client refreshes the access token automatically. End-to-end encryption relies exclusively on the browser's native WebCrypto implementation: ECDH over P-256 for key agreement, HKDF-

SHA256 for key derivation and AES-256-GCM for authenticated encryption. Private identity keys are generated as non-extractable CryptoKey objects and kept in IndexedDB. Media URLs are signed with an HMAC and expire, and TURN credentials are short-lived and derived per user with an HMAC. @fastify/rate-limit protects the API globally and applies a tighter limit to the login route.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

Solace follows a three-tier architecture with an additional peer-to-peer path for calls. Figure 4.1 shows the presentation layer (the Next.js client with its in-browser encryption engine), the application layer (the Fastify API and WebSocket hub) and the data layer (MongoDB with GridFS), and how a request travels between them.



The server stores and routes ciphertext only. Plaintext and private keys exist only inside the browsers.

Reverse proxy (Caddy) terminates TLS in production

Figure 4.1: High-level architecture of Solace

4.2 Description of Each Layer

The Presentation Layer is the Next.js single-page chat application. It performs all encryption and decryption locally using the `@solace/crypto` package, keeps the device identity key pair in IndexedDB, and talks to the server only through HTTPS calls and one WebSocket connection. The server therefore never receives plaintext or private keys.

The Application Layer is the Fastify server. Every protected request passes through an authentication plugin that verifies the JWT and attaches the user's identity; route handlers validate input with Zod, enforce membership and role rules (for example that only members of a direct conversation can call each other), and then read or write data. The WebSocket hub pushes realtime events to connected clients and, through MongoDB change streams, to clients connected to other server instances. Call lifecycle state (ringing, active, ended, busy locks) is tracked here, but the audio and video themselves never touch the server.

The Data Layer is MongoDB running as a replica set. Prisma gives type-safe access to the core models and the transactions they need, the native driver serves notes, decisions, polls and call state, and GridFS stores encrypted media blobs behind signed, expiring URLs. Only ciphertext, public keys and routing metadata are stored.

4.3 Deployment Architecture

For development, the three parts run as separate local processes: the API on port 4000, the Next.js development server on port 3000, and MongoDB on port 27017. For a production-style deployment the repository includes a Caddyfile that reverse-proxies the web app and API and obtains TLS certificates automatically, a coturn configuration (turnserver.conf) for relaying calls across strict NATs, and DEPLOYMENT.md describing two paths: MongoDB Atlas, or a self-hosted MongoDB replica set on the same server, with the API and web app run as Node processes under a process manager. The web client is also deployed publicly on Vercel at <https://solace-lime-eight.vercel.app/>, where it connects to the separately hosted API and MongoDB database.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The schema keeps each fact in one place and expresses relationships through identifier references (conversation members, device owners, message senders, workspace tasks). A hard rule governs the whole design: no field in any model ever stores plaintext message content, plaintext media or a private key. Anything content-shaped, such as a message body, a task title or an event location, is stored as an opaque ciphertext envelope. Integrity is enforced with unique indexes (for example one username per account and one workspace per conversation), compound indexes on the fields that are actually queried (conversation and creation time, workspace and status, workspace and start time), and Zod validation at the API boundary.

5.2 ER Diagram / Database Schema

The core data model is summarised in Figure 5.1. USER owns one or more DEVICE rows, each holding only the public half of a device's identity key. CONVERSATION (type DIRECT, GROUP or CHANNEL) links to its members through CONVERSATION_MEMBER, which carries the role Owner, Admin or Member, and contains MESSAGE rows holding only the encrypted envelope plus routing metadata. A MESSAGE may have a MEDIA_OBJECT describing an encrypted blob in GridFS (storage key, size and a hash of the ciphertext). A GROUP conversation can have exactly one WORKSPACE, which owns TASK (with SUBTASK and attachments), EVENT (with RSVP records) and FOLDER rows. Reactions, stars, pins, blocks, reports and delivery receipts attach to users and messages.

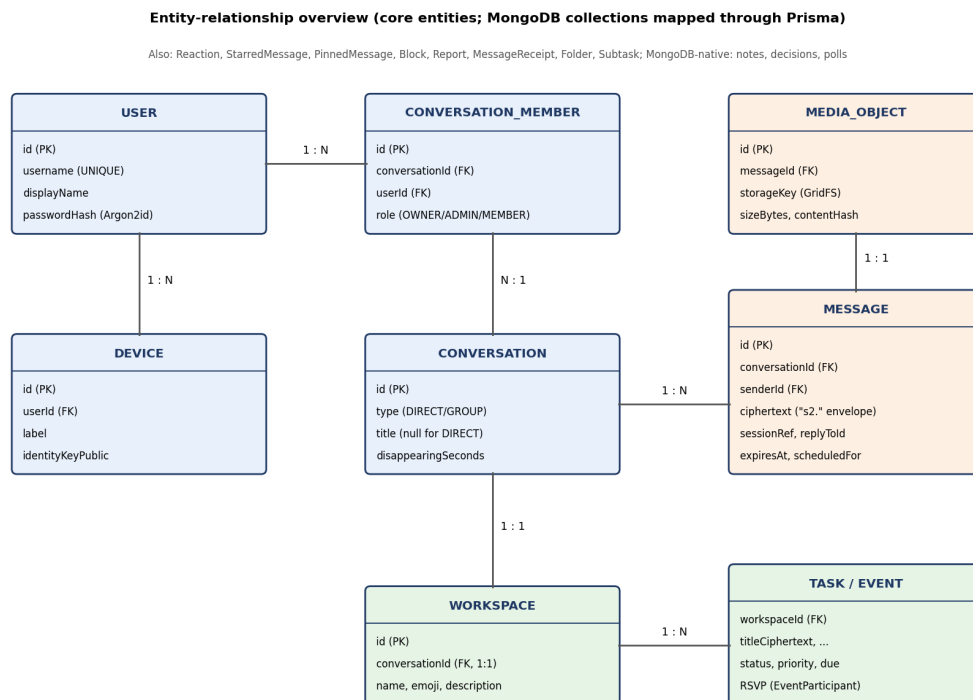


Figure 5.1: Entity-relationship overview of the Solace data model

Document-shaped content that does not need relational queries lives in MongoDB-native collections: notes, decisions and polls (with encrypted text and plain random option identifiers so votes can be tallied without the server learning option text), task comments and activity, and the realtime and call-state collections used for fan-out and call locks.

5.3 Design Decisions and Assumptions

Several decisions were recorded explicitly rather than left implicit in the code:

- Static device keys instead of a ratchet. An earlier version used Olm/Megolm, which advance hidden state on every decrypt; re-renders, multiple tabs or a reload could permanently desynchronise a session and show intermittent “could not decrypt” errors. The design was replaced with a stateless multi-recipient hybrid envelope, in which decryption is a pure function of the ciphertext and the device's static key. The trade-off, stated plainly in the documentation, is that there is no per-message forward secrecy or post-compromise security.
- Recipients are computed per message. Every active device of every conversation member (including the sender's own devices, capped at 256 per message) receives a wrapped copy of the content key, so adding or removing a group member needs no key rotation.
- Metadata the server legitimately needs stays in plaintext: who is talking to whom, timestamps, message sizes, task status, priority and due date, and event start time and timezone. This is the same metadata model as Signal's server; full metadata hiding is out of scope.
- MongoDB as the single datastore. A previous design needed PostgreSQL, Redis, MinIO and MongoDB through Docker. Consolidating on MongoDB (Prisma for core models, change streams for fan-out, GridFS for media, TTL collections for call state) removed three services without changing the API or the client.
- One-to-one calls only. Calls are restricted to direct conversations; the server rejects group call attempts.

5.4 End-to-End Encryption Design

Figure 5.2 summarises how a message is protected. The sender draws a fresh random 256-bit content key and encrypts the plaintext once with AES-256-GCM. For each recipient device, a key-encryption key is derived as HKDF-SHA256 over the ECDH shared secret of the sender's and recipient's device keys, and the content key is wrapped under it with AES-256-GCM. The associated data binds the conversation id, sender device id and recipient device id, so a wrap cannot be replayed into another context. The resulting envelope (the string “s2.” followed by JSON) is sent to the server. A recipient finds its own wrap, derives the same KEK from its private key and the sender's public key, unwraps the content key and decrypts the body.

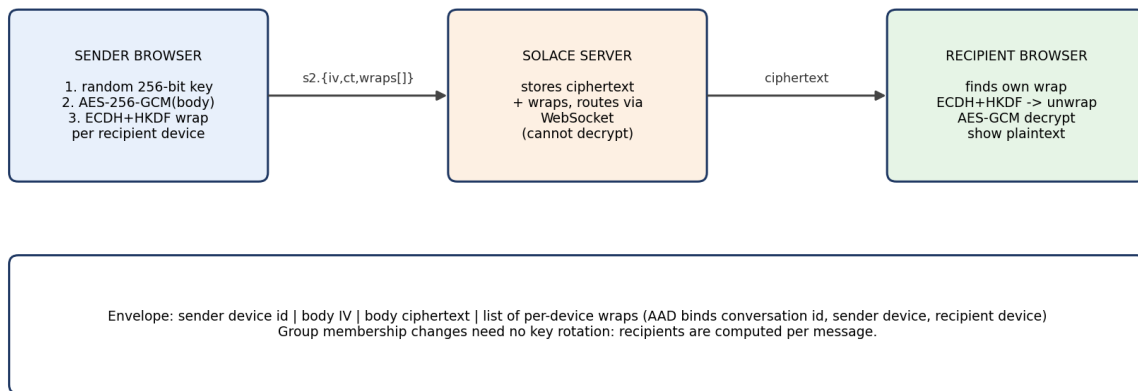


Figure 5.2: End-to-end encrypted message flow

Media follows the same principle: the file is encrypted in the browser with a fresh AES-256-GCM key, the ciphertext is uploaded through a signed expiring URL into GridFS, and the key and IV travel to the recipients inside an ordinary end-to-end encrypted message, never in a URL.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The codebase is a monorepo with four packages. The modules and their responsibilities are:

- packages/crypto: the encryption engine (engine.ts), device identity generation and non-extractable key storage (identity.ts, vault.ts), the device directory with cached, de-duplicated lookups (directory.ts), and the media cipher (mediaCipher.ts). It contains no custom primitives; it only composes WebCrypto.
- Authentication and keys (routes/auth.ts, routes/keys.ts): registration, login, refresh and logout; POST /keys/lookup returns the active devices of the given users or the exact devices requested, which is how a sender learns whom to encrypt for.
- Conversations and messages (routes/conversations.ts, routes/messageActions.ts, routes/groups.ts): direct and group creation, history with pagination, send, schedule, edit, delete, reply, reactions, pins, stars, disappearing-message settings, member management and role changes.
- Media (routes/media.ts, lib/mediaStore.ts): signed upload and download URLs for ciphertext stored in GridFS, with membership checks before download.
- Workspaces (routes/workspaces.ts, tasks.ts, events.ts, notes.ts, decisions.ts, polls.ts, files.ts): the collaboration layer attached one-to-one to a group conversation.
- Realtime and calls (realtime/hub.ts, wsRoute.ts, calls.ts, routes/calls.ts): WebSocket delivery, change-stream fan-out, call lifecycle, busy locks, rate limits and short-lived TURN credentials.
- Jobs (jobs/sweeps.ts): background sweeps that scrub expired disappearing messages and send scheduled messages when due.

The most intricate server logic is call handling. Creating a call takes a busy lock for both users inside a transaction; an unanswered call is reported as missed to both sides after a timer; exactly one accept wins when several devices answer at once; and a lock belonging to a newer call can never be released by an older one. These rules are covered by the call-signaling test suite described in Chapter 8.

6.2 Front-end Logic

The chat page is composed of focused components: a conversation list, a message thread with memoised message rows (so typing in the composer does not re-render the whole history), a members panel, a media bubble that fetches and decrypts on demand with a progress bar, a sticker and GIF picker, and a call overlay. Workspace features appear as tabs (Dashboard, Tasks, Calendar, Files, Notes, Decisions, Polls). Lower-level modules handle the realtime socket with automatic reconnection, per-conversation drafts stored locally, the media pipeline (image re-encoding through a canvas to compress and strip EXIF data, thumbnail generation, encryption and upload with progress), the voice recorder (elapsed timer, discard, five-minute cap, microphone release on exit) and the WebRTC call session. Authentication state is provided through a context (authSession) that refreshes the access token on a timer and retries once on a 401 response.

6.3 API Integration

The web client talks to the API through a single client module (lib/api.ts) that attaches the access token to requests, performs one refresh-and-retry when a request returns 401, and exposes typed helpers to the rest of the application. The server exposes more than 75 REST endpoints across authentication, keys, conversations, groups, messages, media, workspaces, tasks, events, notes,

decisions, polls, files, calls, moderation, link previews and GIF search, plus the /ws WebSocket and the /health and /ready probes (/ready checks MongoDB). Server-side link previews fetch Open Graph tags so the destination site sees the server's address rather than the user's.

6.4 Authentication and Authorization

Registration creates the account, derives an Argon2id password hash and registers the browser's first device by uploading only its public key. Login registers or reuses a device in the same way. The API issues a short-lived access token and sets a rotating refresh token in an HttpOnly cookie. If the server recognises a session but the browser no longer holds the matching private key (for example, site data was cleared), the client ends the session and requires a fresh login, which registers a new device. Authorization is role- and membership-based: only conversation members can read history or download media; Owners and Admins manage members; an Owner can be neither removed nor demoted through the public routes; a blocked user cannot start or continue a direct conversation; and call invites are limited to members of a DIRECT conversation and are rate limited per user.

CHAPTER 7: FEATURES

7.1 List of Main Features

Messaging and privacy features:

- End-to-end encrypted one-to-one and group chat with delivery and read receipts and typing indicators
- Edit, delete, reply, forward, emoji reactions, mentions, pin (conversation-wide) and star (personal)
- Search over the messages already decrypted in the open conversation, and server-side link previews
- Encrypted images, videos, files and voice messages with upload and download progress
- Stickers and GIFs (GIF search through an optional Giphy proxy)
- Disappearing messages and scheduled messages
- Block, unblock and report
- Encrypted one-to-one voice and video calls with call history in the chat

Workspace features (any group conversation can be converted into a workspace):

- Task board with status, priority, due date, labels, assignee, subtasks, attachments and comments, with a confetti animation on completion
- Shared calendar with agenda and month views and Going / Maybe / Can't-go RSVPs
- Notes, an append-only decisions log, and single-choice polls with live tallies
- File library that groups every shared file by category, shows storage usage and supports folders
- Quick actions on any message: Create Task, Create Event and Create Poll

7.2 How Each Feature Works

Sending a message: the client looks up the active devices of every member, encrypts the plaintext once, wraps the content key for each device and posts the envelope. The server stores the ciphertext, creates delivery records and pushes the event over WebSocket. Receivers decrypt locally; because decryption is stateless, it can be repeated or run from several tabs with the same result, and the sender can read their own messages on any of their devices.

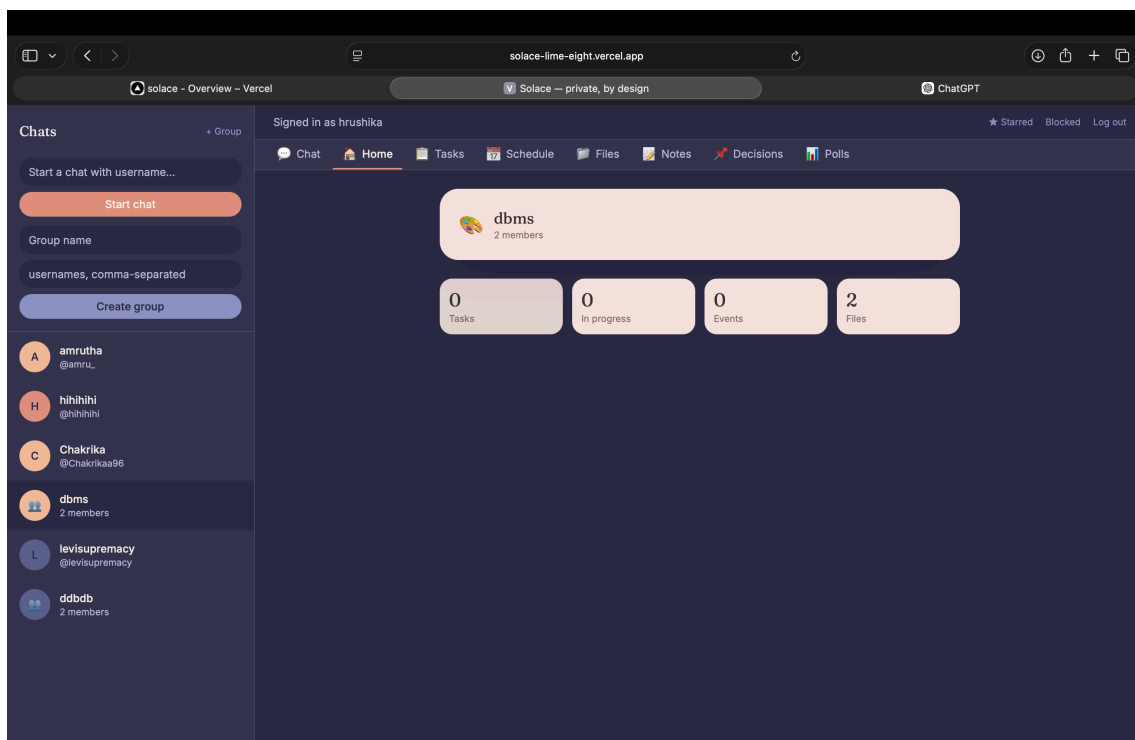
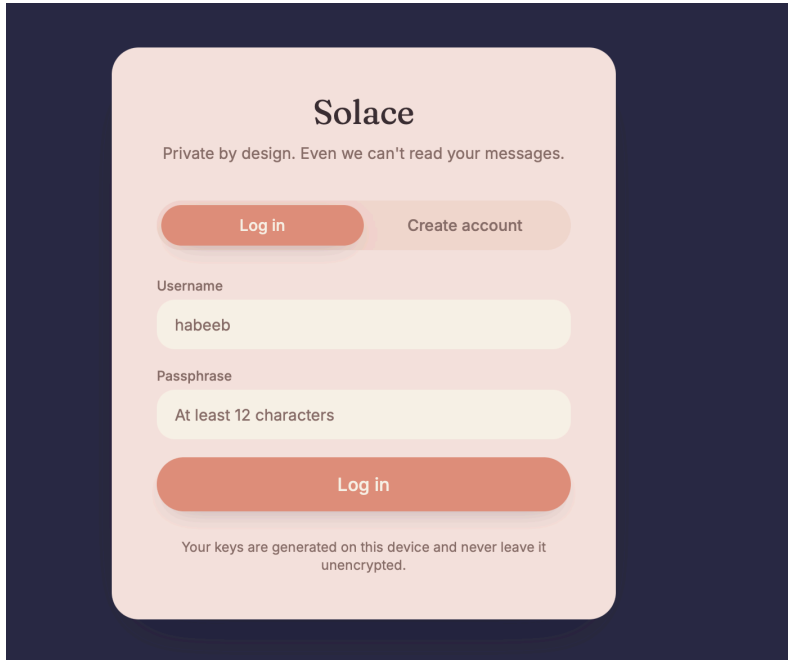
Group membership changes: because recipients are computed per message, a newly added member can read messages sent after joining, a removed member cannot read anything sent after removal, and a device registered later cannot read messages that were sent before it existed.

Disappearing and scheduled messages: each conversation can set `disappearingSeconds`; new messages get an expiry time and a background sweep scrubs the ciphertext and tombstones the row. A scheduled message is encrypted at schedule time, so the server only holds ciphertext and sends it when due. The interface discloses that the server cannot force deletion on a device that has already decrypted and stored a message.

Calls: media flows peer to peer over WebRTC (DTLS-SRTP), or through a TURN relay that sees only ciphertext. The SDP offer and answer, ICE candidates and mute or camera state are encrypted by the clients with the same engine as messages and relayed as opaque frames, so the server cannot alter the DTLS fingerprint and sit in the middle of a call. When a call ends, the caller's client posts one encrypted call message (outcome and duration) so the history appears exactly once.

Workspace content: task titles and descriptions, event titles, descriptions, locations and links, and note, decision and poll text are encrypted in the client before they reach the server. Status, priority,

due date, labels, assignee, event times and recurrence rule stay as plaintext metadata so the server can build the dashboard, filter the board and answer calendar range queries. Each poll option has a random identifier alongside its encrypted text, which lets the server tally votes without learning what any option says.



CHAPTER 8: TESTING

8.1 Automated Unit and Integration Testing

The project contains 60 automated tests written with the Node.js built-in test runner. The crypto package has 22 tests covering the protocol (round trips, repeated and concurrent decryption, multi-device, group membership changes, tamper and replay rejection, directory outages), the device directory cache and the local key vault. The server has 38 tests: 23 for call signaling, 10 for the null-parity layer that makes Prisma on MongoDB write explicit nulls, and 5 for signed media URLs.

8.2 Integration Testing

The call-signaling suite runs against a real MongoDB replica set, started in-process by `mongodb-memory-server` or supplied through `MONGODB_TEST_URL`, so transactions, TTL expiry and atomic updates are exercised against the real database rather than mocks. Beyond the automated suites, the system was checked with a full TypeScript type-check of the server, a production next build of the web application, and manual end-to-end testing with two browser profiles signed in as different users, which is the only way to exercise two-party media. A continuous-integration workflow repeats the crypto tests, server tests, type-check and web build on every push.

8.3 Tools Used for Testing

- Node.js test runner with `tsx` — unit and integration tests
- `mongodb-memory-server` — in-process MongoDB replica set for server tests
- `fake-indexeddb` — IndexedDB emulation for the key-vault tests
- TypeScript compiler (`tsc --noEmit`) and next build — static and build verification
- `curl`, browser developer tools and MongoDB Compass — manual API checks and data inspection
- GitHub Actions — continuous integration

8.4 Test Cases and Results

A representative sample of the automated test cases is shown below. The crypto and server unit tests listed here were executed for this report and all passed (37 of 37 in the crypto, null-parity and media-URL suites); the call-signaling cases run against a MongoDB replica set through `npm test -w @solace/server`.

#	Test Case	Expected Result	Actual Result
1	Recipient and sender both decrypt a message	Both read the same plaintext	Pass
2	Same ciphertext decrypted repeatedly and concurrently	Identical result every time	Pass
3	Message decrypted after a simulated reload, in any order	Decrypts correctly	Pass
4	Device added after a message was sent	Cannot read earlier messages, reads later ones	Pass
5	Group member removed	Removed member cannot read new messages	Pass

#	Test Case	Expected Result	Actual Result
6	Member tries to forge a message from another member	Rejected (sender authentication)	Pass
7	Ciphertext replayed into another conversation	Rejected (bound to conversation)	Pass
8	Body or a key wrap is tampered with	Tampering detected	Pass
9	Key directory unavailable	Decrypt is retryable and recovers; encrypt fails loudly	Pass
10	Identity key saved to and loaded from the vault	Survives round trip, stays non-extractable	Pass
11	Signed upload URL: expired or tampered token	Rejected	Pass
12	Signed URL used for a different operation	Rejected (token bound to its operation)	Pass
13	Prisma create with unset optional fields	Filled with explicit nulls, caller values never overwritten	Pass
14	Call invite to a group or non-member	Refused	Run on replica set
15	Two devices accept the same call at once	Exactly one accept wins	Run on replica set
16	Block between two users, then call	Reported as generic “unavailable”, callee never rings	Run on replica set
17	Unanswered call	Reported missed to both sides, both users freed	Run on replica set
18	Too many call invites from one user	Rate limited	Run on replica set

Testing also shaped the design: the intermittent “could not decrypt” failures seen with the earlier ratchet-based scheme were traced to repeated decrypts advancing hidden state, which led to the stateless envelope design and to tests that deliberately decrypt the same ciphertext many times, in parallel and after a reload. Chapter 10 discusses this further.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

Local development setup:

- Start MongoDB as a replica set (replication.replSetName: rs0 in mongod.conf), or create a free MongoDB Atlas cluster.
- Run `npm install` at the repository root, then copy `apps/server/.env.example` to `apps/server/.env` and `apps/web/.env.example` to `apps/web/.env.local`, and generate two different JWT secrets with `openssl rand -base64 48`.
- Run `npm run prisma:generate` and `npm run db:setup` (initialises the replica set and creates collections and indexes; safe to re-run).
- Start the API with `npm run dev:server` (port 4000) and the web client with `npm run dev:web` (port 3000).
- Open `http://localhost:3000`, create an account, and confirm the API is healthy with `curl http://localhost:4000/ready`.
- Live deployment: the web client is deployed on Vercel and can be opened at <https://solace-lime-eight.vercel.app/> without any local setup.

9.2 Environment Configuration

Server configuration lives in `apps/server/.env`, which is never committed; a `.env.example` with placeholders is committed instead. The main variables are `MONGODB_URL` (connection string with the database name `solace`), `JWT_ACCESS_SECRET` and `JWT_REFRESH_SECRET` (two different secrets), `WEB_ORIGIN` (the web client's URL, used for CORS), `PORT`, and optional `GIPHY_API_KEY`, `PUBLIC_API_URL` and `MEDIA_URL_SECRET`. Voice and video calls use `STUN_URLS`, `TURN_URLS` and `TURN_SECRET`; leaving the `TURN` values empty runs `STUN-only`, which works on local networks and permissive NATs. The web client needs a single variable, `NEXT_PUBLIC_API_URL`, which for the deployed site is set to the public address of the hosted API, while the API's `WEB_ORIGIN` is set to the Vercel address so that CORS and cookies work.

9.3 Hosting of Frontend and Backend

For production, `DEPLOYMENT.md` describes building the API (`npm run build`, then `node dist/index.js`) and the web app (`next build`, then `next start`) and running both under a process manager on one server. The provided Caddyfile reverse-proxies the web app and the API, provisions and renews TLS certificates automatically, and proxies WebSockets without extra configuration. MongoDB can be an Atlas cluster or a self-hosted replica set bound to `127.0.0.1` with authorization enabled and scheduled `mongodump` backups, which include the GridFS media. For calls across strict NATs and mobile networks, a coturn TURN relay is configured with the supplied `turnserver.conf`, using the shared secret that the API uses to derive short-lived credentials.

Live deployment: the Next.js web client is hosted on Vercel and is publicly available at <https://solace-lime-eight.vercel.app/>. The login page of the deployed site loads over HTTPS and offers Log in and Create account, and states that keys are generated on the user's own device. Because the Fastify API keeps a long-lived WebSocket connection for realtime messaging and calls, it runs as a persistent Node process on a separate host (API host: `localhost:4000`), connected to a MongoDB

replica set (MongoDB Atlas / host: vastra.wr8fwex.mongodb.net). A reviewer can open the link, register two accounts in two browser profiles, and try encrypted messaging and the Workspace features directly.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Intermittent “could not decrypt” errors. The original Olm/Megolm design advanced a ratchet on every decrypt, so re-renders, reopened conversations, several tabs or a reload could corrupt a session irreversibly.
- Stale local sessions and lost identities. Early versions minted a new device identity at every login, which orphaned every session established with the previous identity.
- Build-time failures that only appeared in a real build or browser: .js-suffixed workspace imports that webpack could not resolve, and a WebAssembly binary that could not be fetched from hashed chunk paths.
- MongoDB adaptation. Prisma on MongoDB omits unset optional fields instead of storing null, so queries such as deletedAt: null stopped matching; and Prisma transactions and change streams require a replica set.
- Running four services (PostgreSQL, Redis, MinIO, MongoDB) through Docker made local setup heavy.

10.2 Solutions Applied

The cryptographic layer was redesigned rather than patched: the ratchet-based scheme was replaced with a stateless multi-recipient hybrid envelope on WebCrypto, so decryption can be repeated, reordered and run concurrently. The identity is a non-extractable CryptoKey in IndexedDB, restored on reload with no passphrase. A null-parity layer (nullParity.ts) writes explicit nulls on every create, including nested creates, and is verified by tests against the real schema. Bundling issues were fixed with webpack extensionAlias configuration in next.config.js. Replacing PostgreSQL, Redis and MinIO with MongoDB (Prisma for core data, change streams for fan-out, GridFS for media, TTL collections for call state) removed Docker from the setup entirely. The README records each of these problems and its root cause.

10.3 Current Limitations or Known Bugs

- Static device keys give no per-message forward secrecy and no post-compromise security; an attacker who records ciphertext and later obtains a device's private key could read that device's history.
- The server necessarily sees metadata: who talks to whom, when, message sizes, device public keys, and call lifecycle.
- Message search covers only what the browser has already decrypted in the open conversation, because the server cannot index plaintext.
- Group calls are not supported, and a call leaves no history entry if the chat page is closed when it ends.
- Clearing site data removes the device key and requires a fresh login; messages sent before the new device existed stay unreadable on it.
- Several workspace features are intentionally basic: polls are single-choice with no expiry, notes have no version history, recurring events store the rule but are not expanded into occurrences, and files have no versioning.
- Voice-message playback depends on the receiving browser supporting the sender's recording format, because the server cannot transcode ciphertext.
- The 23 call-signaling tests and two-party media must be run against a MongoDB replica set and two browsers; they are not part of a quick local check.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- A forward-secure ratcheting layer on top of the stateless envelope, giving per-message forward secrecy without reintroducing fragile hidden state.
- Group voice and video calls, and true broadcast-only channels with a poster/subscriber distinction.
- Safety-number or key-fingerprint verification screens so users can confirm each other's device keys.
- Cross-device draft and read-state sync through an encrypted channel, and a device-linking flow.
- Natural-language date parsing for the Create Event quick action, week view and expansion of recurring events.

11.2 Possible Integrations or Optimizations

- Push notifications and a progressive web app or native mobile client.
- Containerising the application with Docker for reproducible deployment, and adding end-to-end browser tests (for example with Playwright) to automate the two-browser call scenarios.
- Streaming, chunked encryption for very large files so they need not be held in memory, with resumable uploads.
- Moving to managed MongoDB Atlas search and monitoring, and caching device-directory lookups more aggressively under heavy load.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to build a messaging platform in which privacy is the default rather than an option. Solace is a TypeScript monorepo consisting of a Next.js client, a Fastify API with a WebSocket hub, a standalone WebCrypto-based encryption package and a design-token package, all backed by a single MongoDB replica set. Over seven build phases and three follow-on feature systems, it grew from an authentication screen into a complete encrypted messenger with collaboration tools.

12.2 What Was Achieved

The finished system delivers end-to-end encrypted one-to-one and group chat in which the server only ever stores ciphertext; encrypted media and voice messages; the full set of message features; disappearing and scheduled messages; blocking and reporting; end-to-end encrypted signaling for peer-to-peer voice and video calls; and a Workspace layer with tasks, calendar, notes, decisions, polls and files. A cryptographic redesign removed a class of intermittent decryption failures, the infrastructure was reduced to a single datastore, and the work is backed by 60 automated tests, a CI pipeline and deployment documentation. The web client is deployed publicly at <https://solace-lime-eight.vercel.app/>.

12.3 Skills Learned

The project gave practical experience in applied cryptography with WebCrypto (key agreement, key derivation, authenticated encryption and why composition choices matter); threat modelling and being explicit about what a design does and does not protect; real-time systems using WebSockets, change streams and WebRTC signaling; NoSQL data modelling and transactions with Prisma and MongoDB; secure authentication with Argon2id, JWT and refresh-token rotation; full-stack TypeScript with Next.js and Fastify; and systematic debugging, in which finding the root cause of the decryption failures led to a better design instead of a workaround.

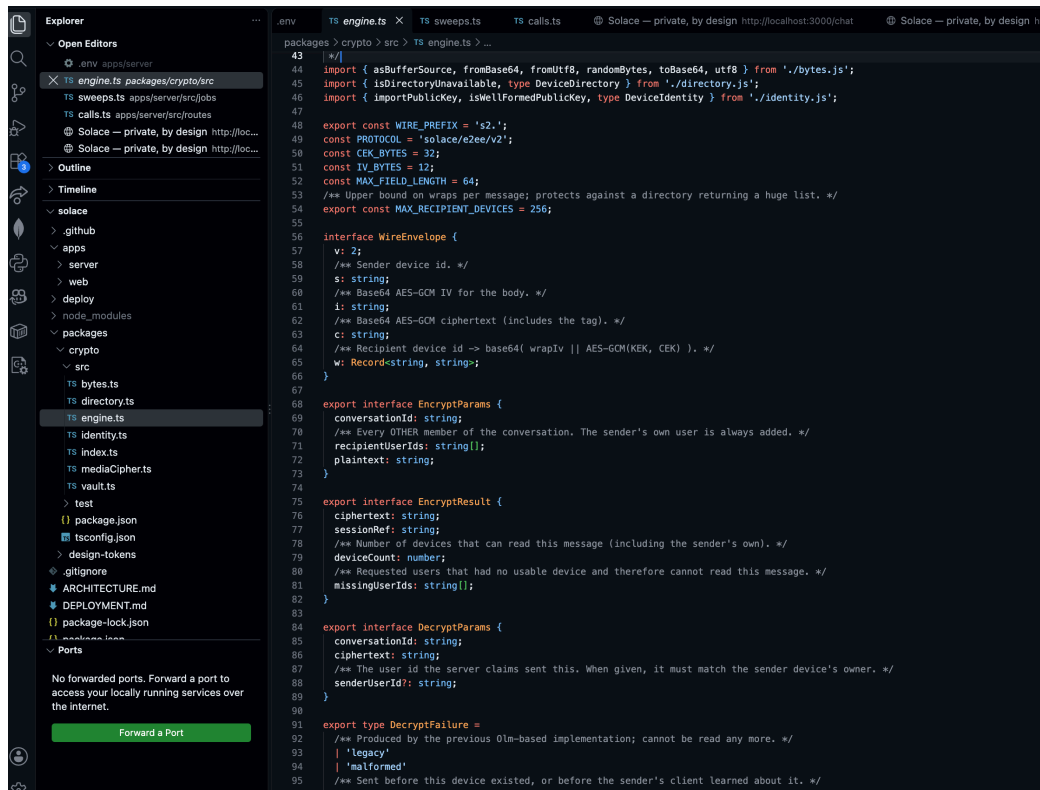
CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev>
- Next.js — Official Documentation (App Router), <https://nextjs.org/docs>
- Fastify — Official Documentation, <https://fastify.dev>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- MongoDB Manual (replica sets, change streams, GridFS), <https://www.mongodb.com/docs/manual/>
- Prisma — MongoDB connector documentation, <https://www.prisma.io/docs/orm/overview/databases/mongodb>
- MDN Web Docs — Web Crypto API (SubtleCrypto), https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API
- MDN Web Docs — WebRTC API, https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API
- NIST SP 800-38D — Recommendation for GCM and GMAC, <https://csrc.nist.gov/pubs/sp/800/38/d/final>
- RFC 5869 — HMAC-based Extract-and-Expand Key Derivation Function (HKDF), <https://www.rfc-editor.org/rfc/rfc5869>
- OWASP Password Storage Cheat Sheet (Argon2id), https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- jsonwebtoken — npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- Tailwind CSS — Official Documentation, <https://tailwindcss.com/docs>
- Zod — Official Documentation, <https://zod.dev>
- coturn TURN server — Project page, <https://github.com/coturn/coturn>
- Caddy — Reverse proxy and automatic HTTPS, <https://caddyserver.com/docs/>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

Derivation of the key-encryption key (KEK) used to wrap the message content key for one recipient device, from packages/crypto/src/engine.ts. ECDH produces a shared secret, which HKDF-SHA256 turns into a non-extractable AES-256-GCM key bound to the sender and recipient device ids:



```
43 |  
44 |  
45 | import { asBufferSource, fromBase64, fromUtf8, randomBytes, toBase64, utf8 } from './bytes.js';  
46 | import { isDirectoryUnavailable, type DeviceDirectory } from './directory.js';  
47 | import { importPublicKey, isWellFormedPublicKey, type DeviceIdentity } from './identity.js';  
48 |  
49 | export const WIRE_PREFIX = 's2';  
50 | const PROTOCOL = 'solace/e2ee/v2';  
51 | const CK_BYTES = 32;  
52 | const IV_BYTES = 12;  
53 | const MAX_FIELD_LENGTH = 64;  
54 | /** Upper bound on wraps per message; protects against a directory returning a huge list. */  
55 | export const MAX_RECIPIENT_DEVICES = 256;  
56 |  
57 | interface WireEnvelope {  
58 |   v: 2;  
59 |   /** Sender device id. */  
60 |   s: string;  
61 |   /** Base64 AES-GCM IV for the body. */  
62 |   i: string;  
63 |   /** Base64 AES-GCM ciphertext (includes the tag). */  
64 |   c: string;  
65 |   /** Recipient device id -> base64( wrapIV || AES-GCM(KEK, CK) ). */  
66 |   w: Record<string, string>;  
67 | }  
68 |  
69 | export interface EncryptParams {  
70 |   conversationId: string;  
71 |   /** Every OTHER member of the conversation. The sender's own user is always added. */  
72 |   recipientUserIds: string[];  
73 |   plaintext: string;  
74 | }  
75 |  
76 | export interface EncryptResult {  
77 |   ciphertext: string;  
78 |   sessionRef: string;  
79 |   /** Number of devices that can read this message (including the sender's own). */  
80 |   deviceCount: number;  
81 |   /** Requested users that had no usable device and therefore cannot read this message. */  
82 |   missingUserIds: string[];  
83 | }  
84 |  
85 | export interface DecryptParams {  
86 |   conversationId: string;  
87 |   ciphertext: string;  
88 |   /** The user id the server claims sent this. When given, it must match the sender device's owner. */  
89 |   senderUserId?: string;  
90 | }  
91 |  
92 | export type DecryptFailure =  
93 |   | 'legacy'  
94 |   | 'malformed'  
95 |   /** Sent before this device existed, or before the sender's client learned about it. */  
96 |   | 'not found for this device'
```

The content key is then wrapped for each recipient device, with the conversation id, sender device id and recipient device id bound in as additional authenticated data:

Appendix B: Installation and Setup Instructions

- Install Node.js 20+ and MongoDB 5.0+; configure MongoDB as a replica set (replication.replSetName: rs0) or use a MongoDB Atlas connection string.
- npm install
- cp apps/server/.env.example apps/server/.env and cp apps/web/.env.example apps/web/.env.local; set JWT_ACCESS_SECRET and JWT_REFRESH_SECRET using openssl rand -base64 48 (run twice).
- npm run prisma:generate
- npm run db:setup
- npm run dev:server (API on http://localhost:4000) | npm run dev:web (web on http://localhost:3000)
- Optional: npm test -w @solace/crypto and npm test -w @solace/server to run the automated tests.
- Live deployment (no setup needed): open <https://solace-lime-eight.vercel.app/>.

Appendix C: Default Login Credentials

Solace does not ship with seeded or default accounts, because every account must generate its own encryption keys in the browser at registration. To try the system, open the deployed site at <https://solace-lime-eight.vercel.app/> (or <http://localhost:3000> when running locally), create two accounts (for example “alice” and “bob”) in two different browser profiles, start a direct conversation by username, and send messages. Create a group to try membership changes and the Workspace features. The first browser to register an account becomes that account’s device; clearing site data removes the device key and requires logging in again.

Appendix D: API Reference Summary

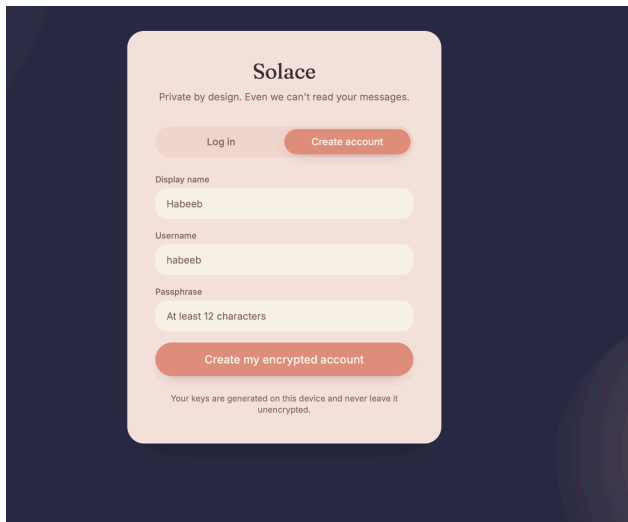
More than 75 REST endpoints plus one WebSocket endpoint, grouped by resource. All routes except registration, login and refresh require a valid access token.

Resource	Endpoints
Auth	POST /auth/register, POST /auth/login, POST /auth/refresh, POST /auth/logout, GET /auth/me
Keys and Users	POST /keys/lookup, GET /users/lookup, POST /users/:id/block, DELETE /users/:id/block, GET /me/blocked
Conversations	POST /conversations/direct, POST /conversations/group, GET /conversations, GET POST /conversations/:id/messages, POST /conversations/:id/messages/schedule, PATCH /conversations/:id/disappearing, GET /conversations/:id/pins
Members	GET POST /conversations/:id/members, PATCH DELETE /conversations/:id/members/:userId
Message actions	PATCH DELETE /messages/:id, POST /messages/:id/reactions, POST DELETE /messages/:id/pin, POST DELETE /messages/:id/star, GET /me/starred, POST /messages/:id/report
Media	POST /media/upload-url, GET /media/:messageId/download-url, PATCH /media/:messageId/folder
Workspaces	POST GET /conversations/:id/workspace, GET /workspaces/:id/dashboard, PATCH /workspaces/:id
Tasks	POST GET /workspaces/:id/tasks, GET PATCH DELETE /tasks/:id, POST /tasks/:id/subtasks, PATCH DELETE /subtasks/:id, POST /tasks/:id/attachments, GET /tasks/:id/activity, POST /tasks/:id/comments
Events	POST GET /workspaces/:id/events, GET PATCH DELETE /events/:id, POST /events/:id/rsvp
Notes, Decisions, Polls	POST GET /workspaces/:id/notes, PATCH DELETE /notes/:id, POST GET /workspaces/:id/decisions, DELETE /decisions/:id, POST GET /workspaces/:id/polls, POST /polls/:id/vote, DELETE /polls/:id
Files	GET /workspaces/:id/files, GET /workspaces/:id/storage, POST GET /workspaces/:id/folders, PATCH DELETE /folders/:id
Calls	GET /calls/ice-servers (signaling frames call.* travel over /ws)
Utilities	GET /link-preview, GET /gifs/search, GET /health, GET /ready, WebSocket /ws

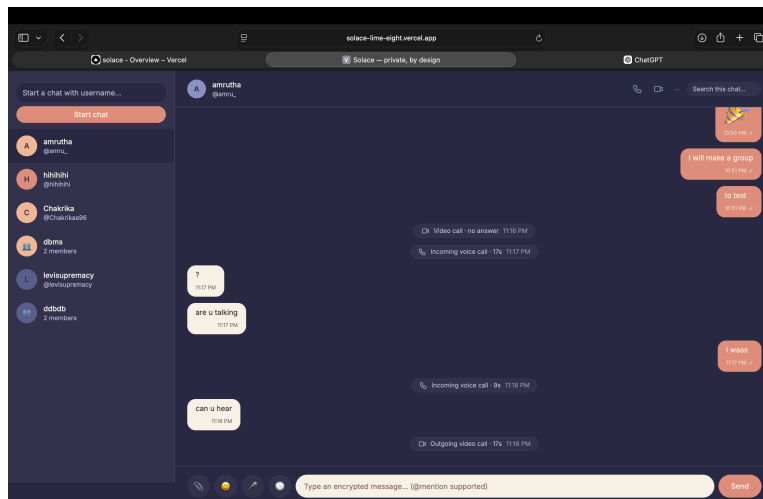
Appendix E: Screenshots

The following pages are left for the team to complete with actual screenshots from the running application before submission. Suggested screenshots, in order:

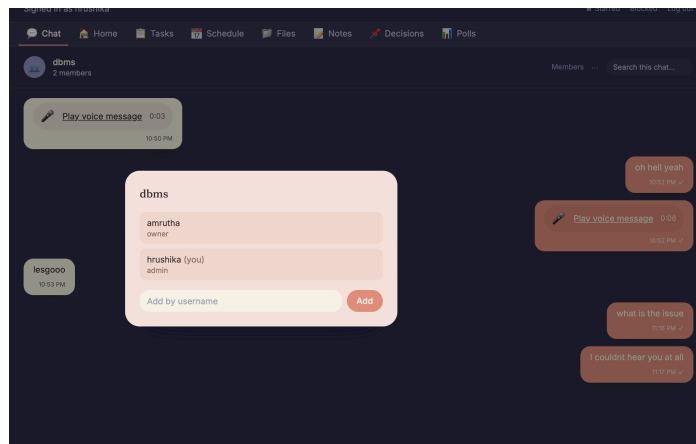
- Registration / Login page (deployed site, <https://solace-lime-eight.vercel.app/>)



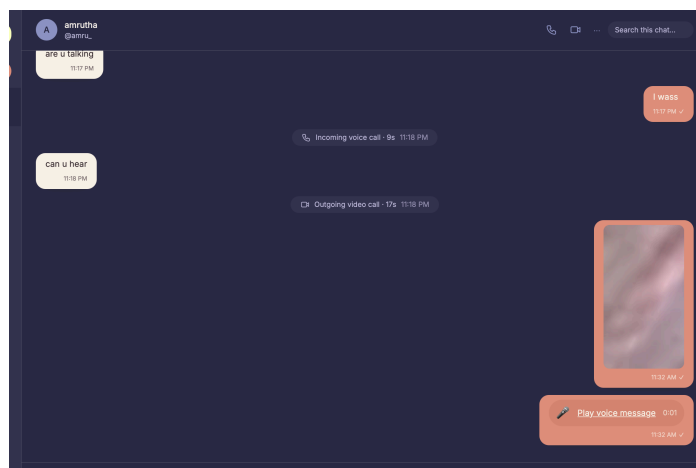
- Conversation list with an open end-to-end encrypted chat



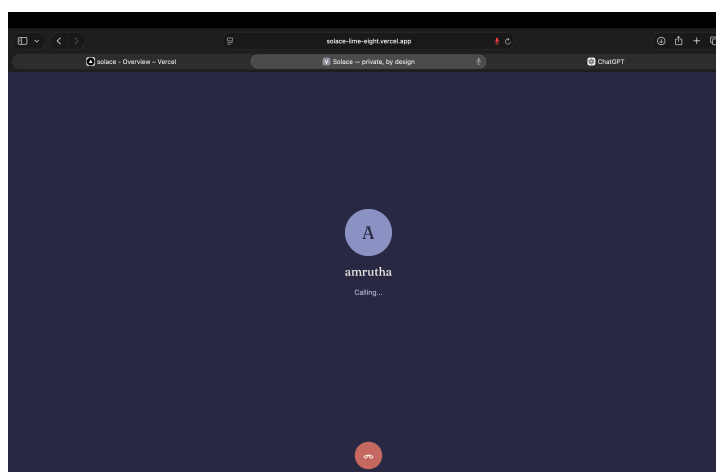
- Group chat with the Members panel



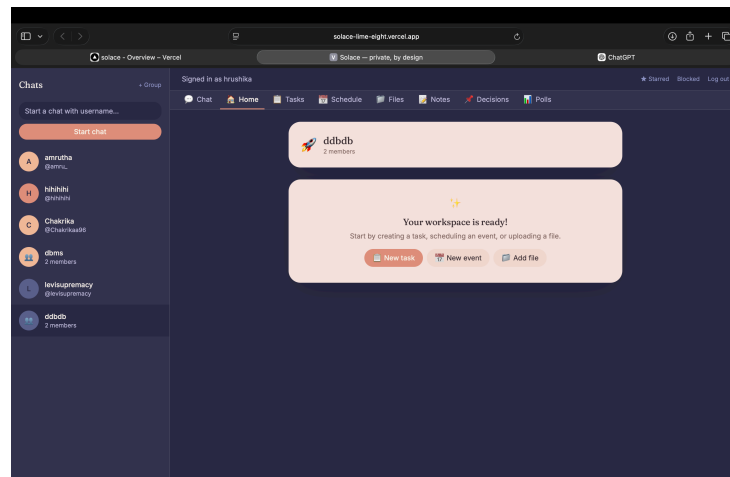
- Message with an image or file and the voice-message recorder



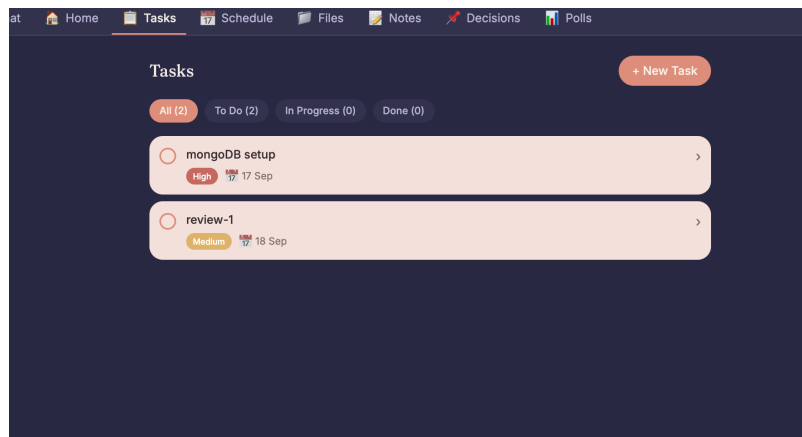
- Voice / video call overlay



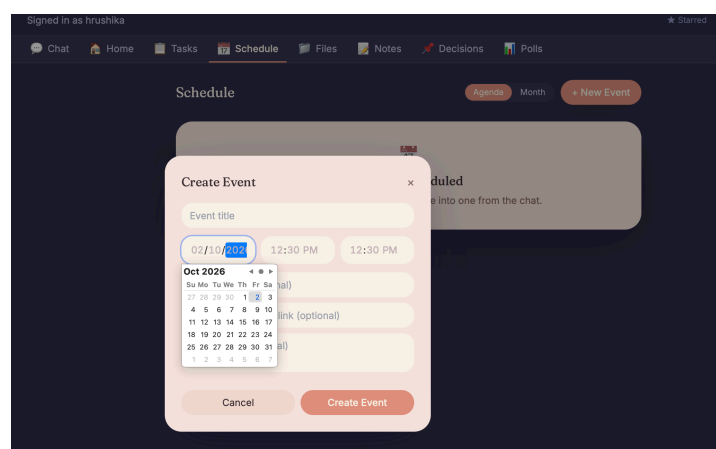
- Workspace Dashboard



- Task Board and Task Detail panel



- Calendar view with RSVP buttons



- Files tab and Polls tab

